



EXPLOITS UNIVERSITY

BSc INFORMATION COMMUNICATION AND TECHNOLOGY

BICT 3202 · OBJECT-ORIENTED ANALYSIS AND DESIGN

WEEK 12

Integrated Object-Oriented Modelling and Design

PROGRAMME	BSc Information Communication and Technology
COURSE CODE / YEAR	BICT 3202 · Year 3
LECTURER	Francis Fweta — ICT Lecturer
DELIVERY	2h Lecture · 1h Tutorial · 1h Practical · 10h Independent Learning

Prepared by Francis Fweta · Exploits University · Week 12 of 16

Contents

1. Week 12 Introduction	4
2. Learning Outcomes	4
PART A — WHAT IS INTEGRATED MODELLING?	5
3. Meaning of Integrated Modelling	5
4. Different UML Models Describe Different Views	5
5. The Big Picture	6
PART B — WHY DO MODELS NEED TO AGREE?	7
6. Model Consistency	7
PART C–E — THE CASE STUDY AND TRACEABILITY	7
7. University Course Registration — Building the Models	7
8. Traceability	8
PART F — THE SEQUENCE DIAGRAM	8
9. Why a Sequence Diagram — and Checking Consistency	8
PART G — STATE MODELLING	9
10. The Registration State Model	9
PART H — ACTIVITY DIAGRAM	10
11. Activity Diagram vs Sequence Diagram	10
PART I — INTEGRATING THE FOUR MODELS	11
12. Putting Everything Together	11
PART J — MODEL CONSISTENCY CHECKING	11
13. The Consistency Checklist	11
PART K — MULTIPLICITY	12
14. Why Multiplicity Matters	12
PART L — PACKAGES: ORGANISING A LARGE MODEL	12
15. Why Packages?	12
PART M & N — ARCHITECTURAL INTEGRATION AND COMPONENTS	13
16. Layered Architecture	13
17. Component Thinking	14
PART O & P — VALIDATION AND DESIGN REVIEW	15



18. Design Validation	15
19. Questions a Design Team Should Ask	15
PART Q — MODEL vs DIAGRAM	16
20. An Important Distinction	16
PART R — THE COMPLETE INTEGRATED EXAMPLE	16
21. Tutorial and Practical	17
22. Common Student Confusions	17
23. Week 12 Summary	18
24. Examination-Style Questions	18
25. References and Further Reading	18

1. Week 12 Introduction

Week 10 asked “What does it mean to move from analysis into design?” Week 11 asked “How do we make our analysis models detailed enough for implementation?” Week 12 now asks the next question: “**How do we bring all these different models together into ONE coherent system design?**”

A real OOAD project does not consist of a class diagram sitting separately from a use-case diagram, sequence diagram and state diagram. They should describe the **same system from different perspectives**. UML is specifically intended for visualising, specifying and documenting software and systems, and the current formal UML specification listed by OMG is UML 2.5.1.

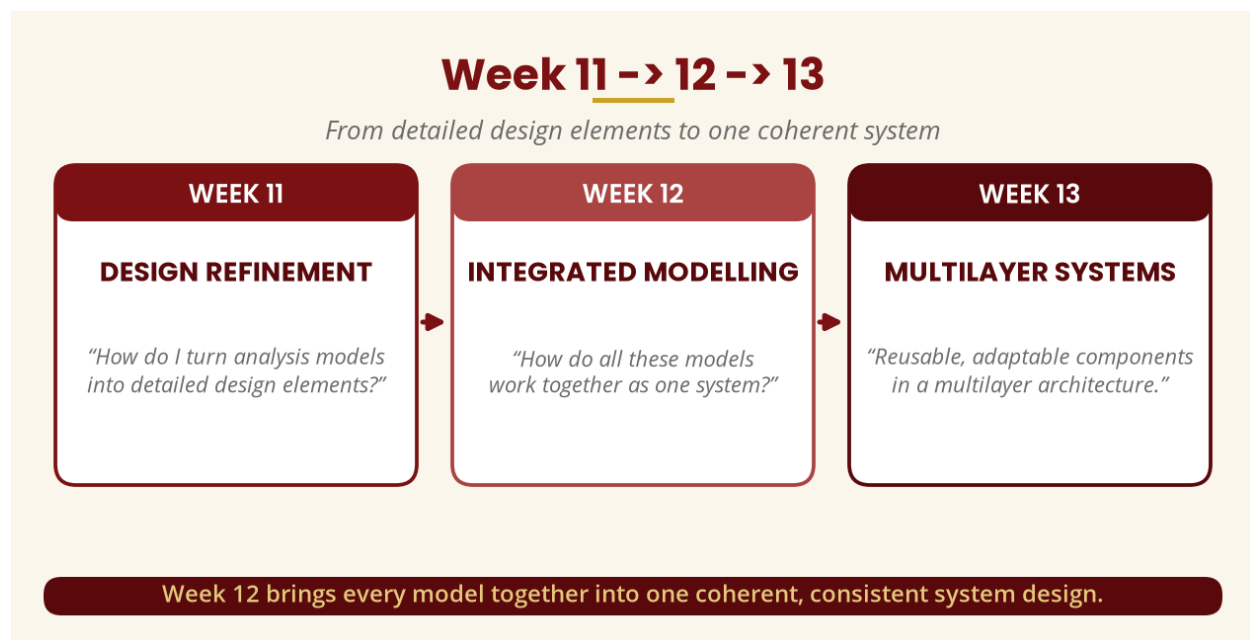


Figure 1 — Week 11 -> 12 -> 13 progression

2. Learning Outcomes

By the end of this week, a student should be able to:

- Explain the meaning of integrated modelling and why models must be consistent.
- Explain the relationship between the use-case, class, interaction, state and activity models.
- Trace a requirement from use cases through classes, interactions and operations.
- Apply multiplicity and packages to organise and detail a model.
- Think at component and architecture level.
- Detect contradictions between models and validate a design against its requirements.
- Distinguish a model from a diagram.
- Build a complete integrated UML model for a realistic system.

PART A — WHAT IS INTEGRATED MODELLING?

3. Meaning of Integrated Modelling

Building a house requires an architectural drawing, an electrical plan, a plumbing plan and a structural plan. Each shows something different, but they must describe the **same house** — you cannot have the architectural drawing say 3 bedrooms while the electrical drawing says 5 and the plumbing drawing says 2. Software models work the same way.

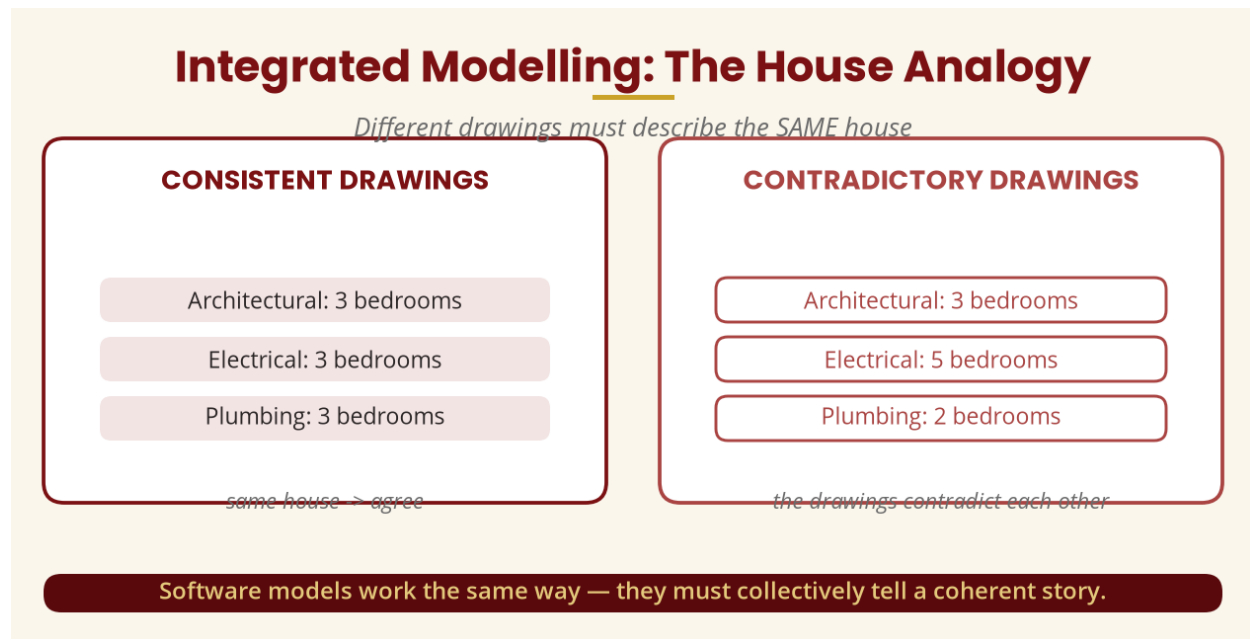


Figure 2 — The house analogy

4. Different UML Models Describe Different Views

The **use-case model** answers “what does the user want to accomplish?” The **class model** answers “what objects/classes make up the system?” The **sequence diagram** answers “how do objects communicate to accomplish a task?” The **state machine** answers “how does an object change state?” The **activity diagram** answers “what activities and decisions occur in a process?” **Component/deployment views** answer “how is the software organised and deployed?” UML groups structural modelling and behavioural modelling into different parts of its specification.

Different Models, Different Questions

Many views of the same system

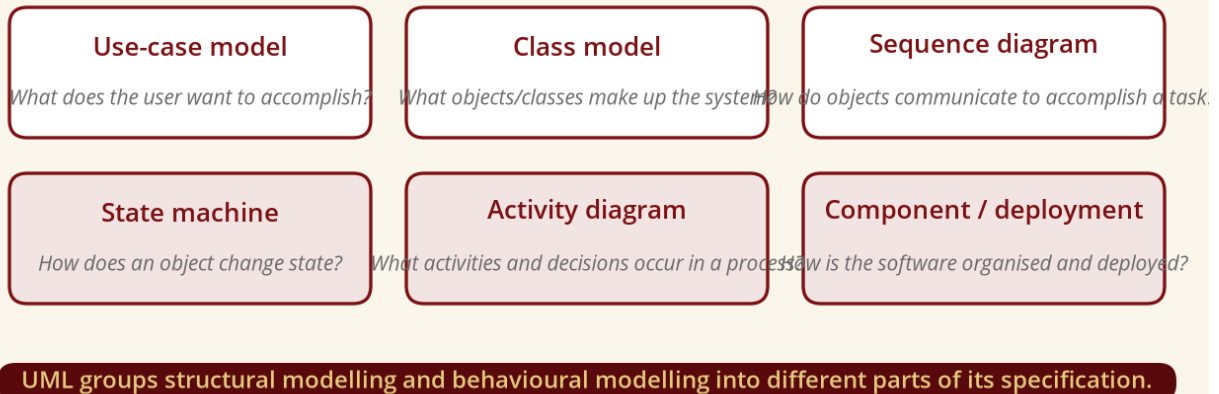


Figure 3 — Different models, different questions

5. The Big Picture

Requirements -> use-case model -> (Object model / Dynamic model / Behavioural model) -> class diagram, state model, sequence/activity model -> integrated design -> implementation.
This is the central idea of Week 12.

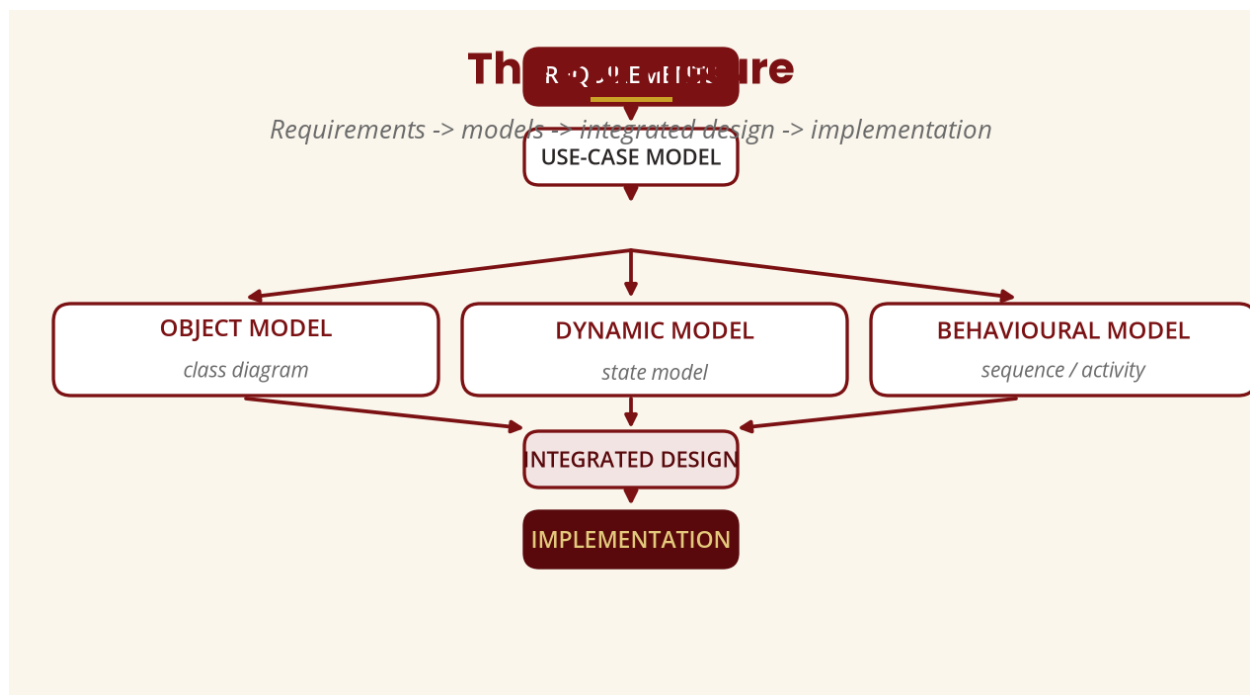


Figure 4 — The big picture

PART B — WHY DO MODELS NEED TO AGREE?

6. Model Consistency

Suppose the requirement says “A student can register for a course.” The use-case diagram contains **Student -> Register for Course** — good. But the class diagram contains only **Student**, **Teacher**, **Department**, with no **Course** or **Registration**. The use-case model says the system must support course registration, but the class model lacks the objects needed to support it — **the models are inconsistent or incomplete**. This is why OOAD is not simply about drawing diagrams: the diagrams must collectively tell a coherent story.

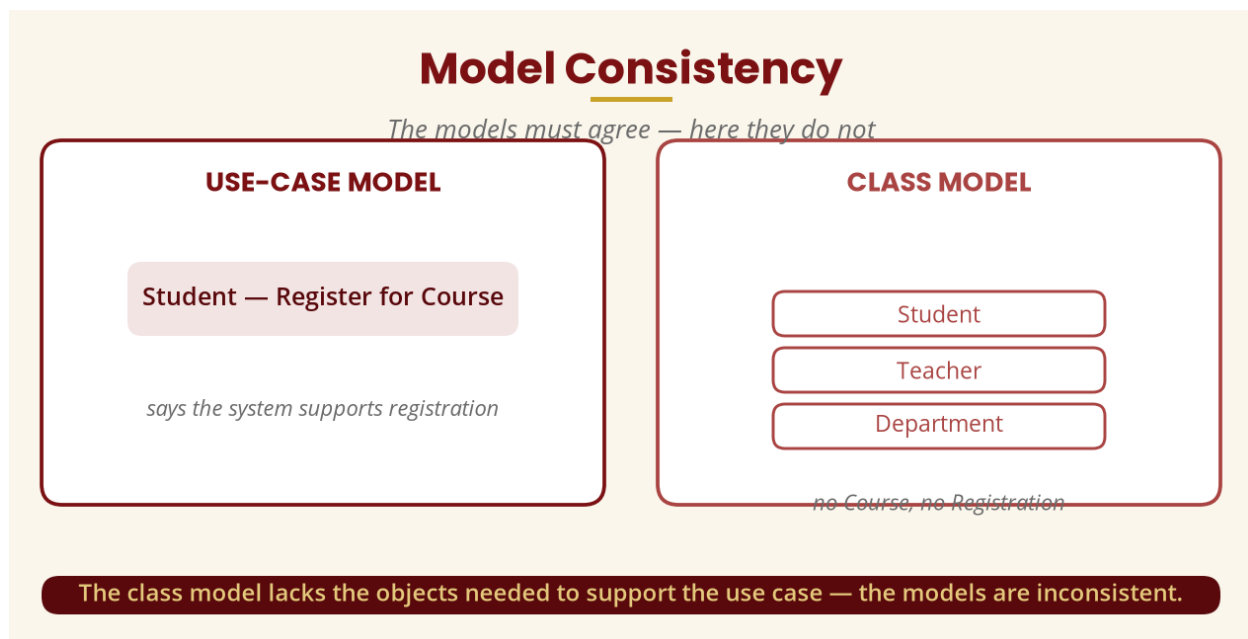


Figure 5 — Model consistency

PART C–E — THE CASE STUDY AND TRACEABILITY

7. University Course Registration — Building the Models

A university wants an online course registration system in which students log in, view available courses, register, drop and view their registered courses; the system checks capacity and prerequisites, stores registrations and notifies the student. **Step 1 — actors:** Student (later Administrator, Lecturer, Finance Officer). **Step 2 — use cases:** Login, View Courses, Register for Course, Drop Course, View Registered Courses. **Step 3 — classes:** domain classes Student, Course, Registration; service classes AuthenticationService, RegistrationService, CourseRepository, RegistrationRepository. **Step 4 — initial class model:** Student (studentId, name, email) — registers -> Registration (registrationId, date, status) — for -> Course (courseCode, title, credits, capacity).

8. Traceability

Traceability means being able to answer “Where did this design element come from?” and “Which requirement does this design element support?” — e.g. Requirement R01 -> Use Case “Register for Course” -> sequence diagram -> classes Student, Course, Registration, RegistrationService -> operations register(), checkCapacity(), checkPrerequisite(). That is a traceable design.

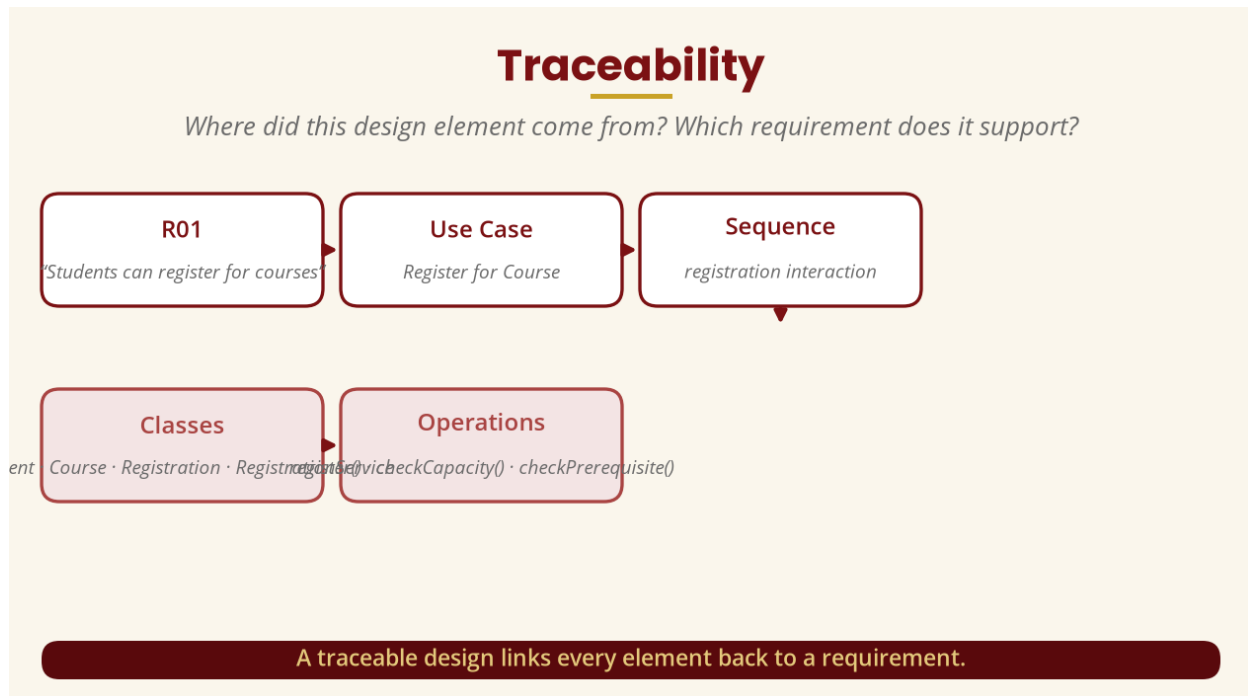


Figure 6 — Traceability

PART F — THE SEQUENCE DIAGRAM

9. Why a Sequence Diagram — and Checking Consistency

The use case says “Student registers for a course” but does not tell us exactly how the interaction happens. The sequence diagram provides that detail: Student -> RegistrationService register(course) -> Course checkCapacity() and checkPrerequisite() -> RegistrationService createRegistration() -> Registration -> confirmation back to Student. Checking consistency: the elements used in the sequence diagram (Student, RegistrationService, Course, Registration) should exist in the class model. If the sequence suddenly says PaymentProcessor but the system has no payment requirement, ask why — maybe a requirement was forgotten, the class is unnecessary, or the model needs updating. That is model validation.

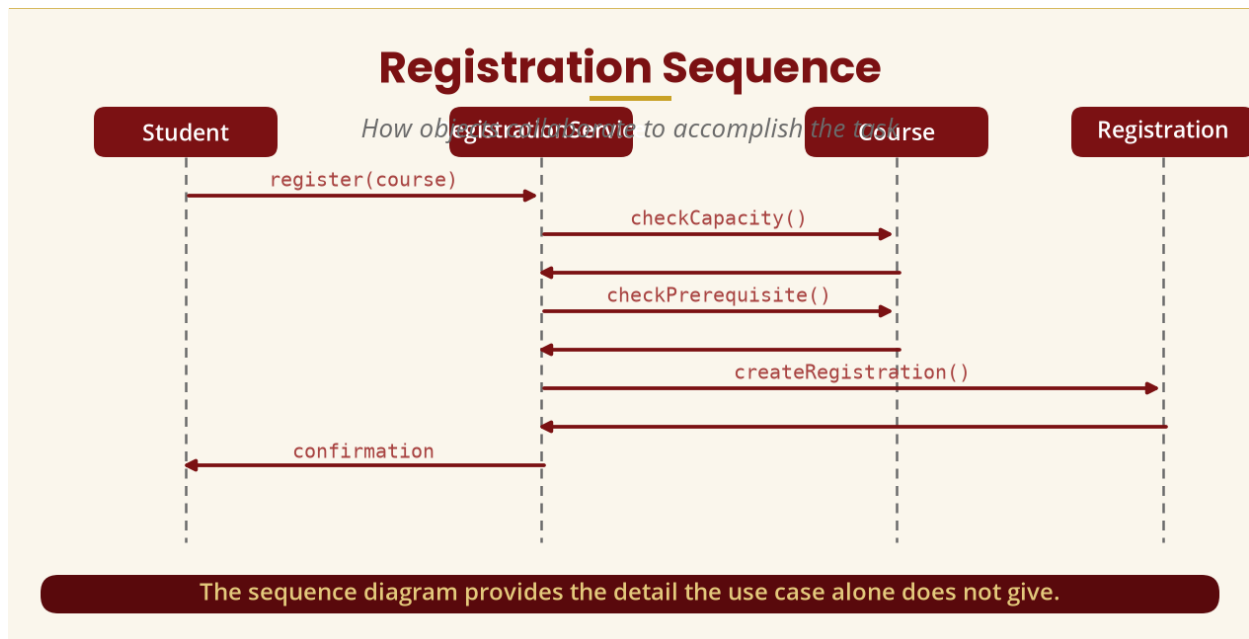


Figure 7 — Registration sequence

PART G — STATE MODELLING

10. The Registration State Model

A registration is not always in the same condition — it may go through Requested, Validated, Confirmed, Cancelled or Rejected. The state model forces the business rules to be explicit: is **[CONFIRMED] -> [CANCELLED]** final, or does the business allow **[CANCELLED] -> [REQUESTED]** -> **[CONFIRMED]** again? The model makes the decision visible.

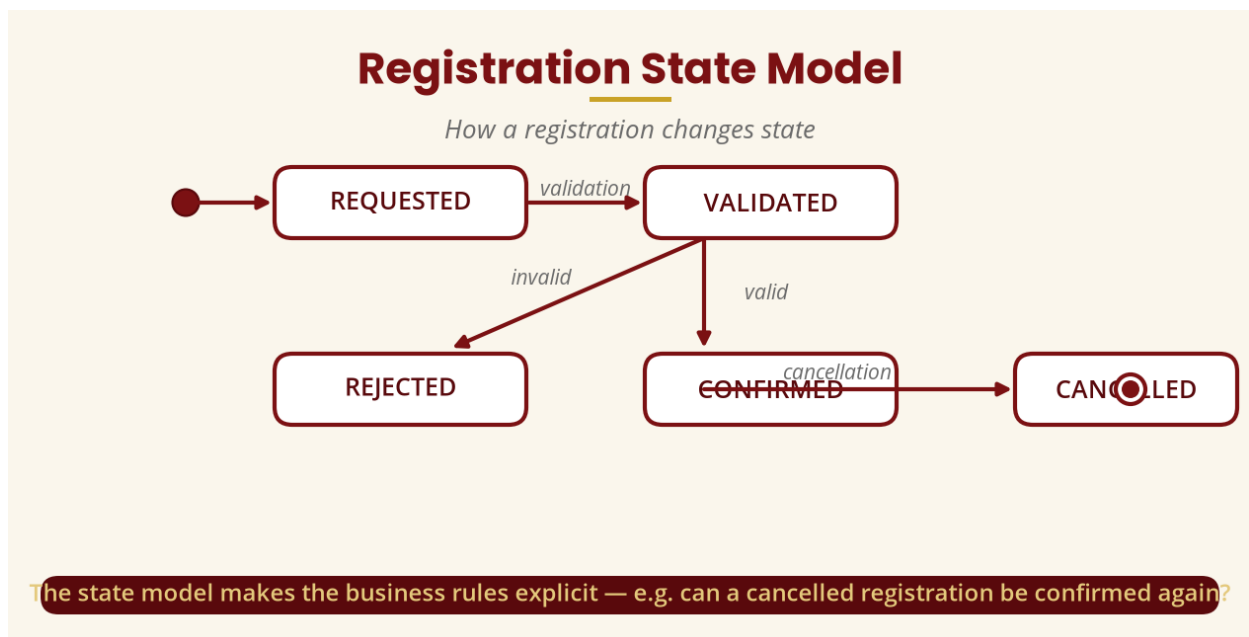


Figure 8 — Registration state model

PART H — ACTIVITY DIAGRAM

11. Activity Diagram vs Sequence Diagram

Students often confuse these. The **sequence diagram** focuses on *who communicates with whom, and in what order*. The **activity diagram** focuses on *what activities occur in a workflow*. The registration activity is: login -> display courses -> select course -> check prerequisites -> [satisfied?] -> check capacity -> [space available?] -> create registration -> confirm.

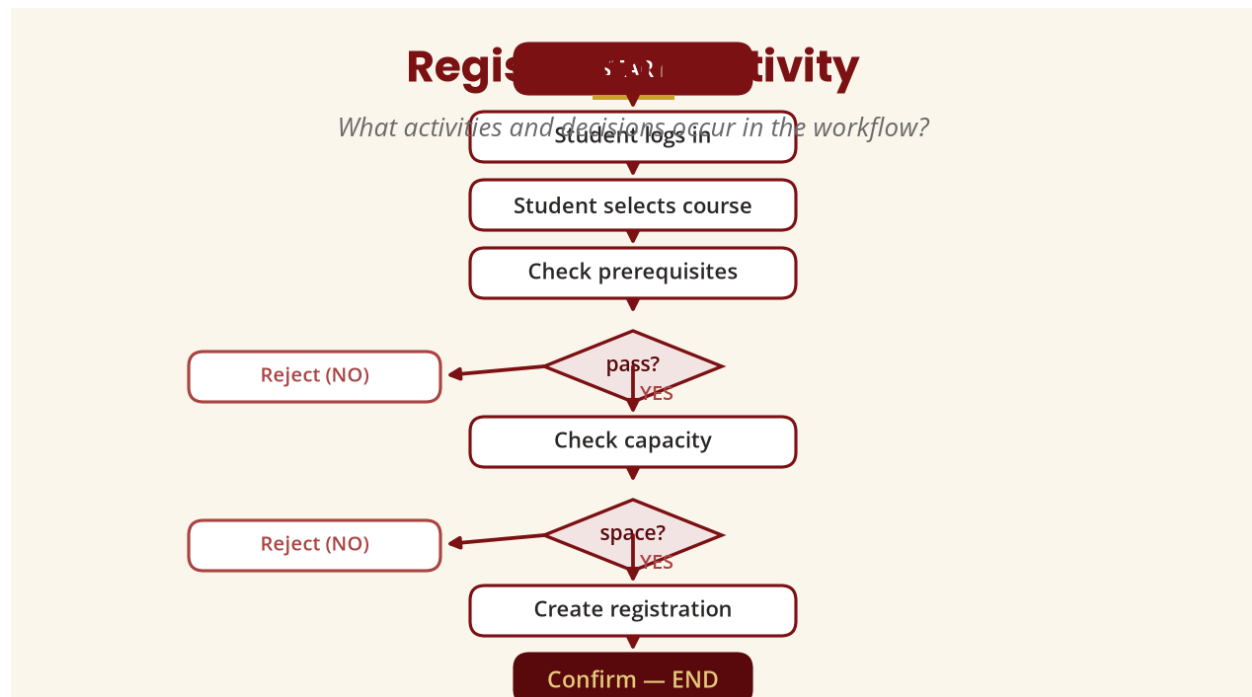


Figure 9 — Registration activity

Comparing the Behavioural Views

A distinction that is extremely important

MODEL	MAIN QUESTION
Use case	What does the actor want to accomplish?
Sequence	How do objects interact over time?
Activity	What workflow / activities take place?
State machine	How does an object change state?

Know which model answers which question before the examination.

Figure 10 — Comparing the behavioural views

PART I — INTEGRATING THE FOUR MODELS

12. Putting Everything Together

For **Register for Course** we now have the use case (Student -> Register for Course), the activity (login -> select course -> check prerequisite -> check capacity -> create -> confirm), the sequence (Student -> RegistrationService -> Course -> Registration), the state (Requested -> Validated -> Confirmed) and the class model (Student, Course, Registration, RegistrationService). These models now describe the same functionality from different viewpoints.

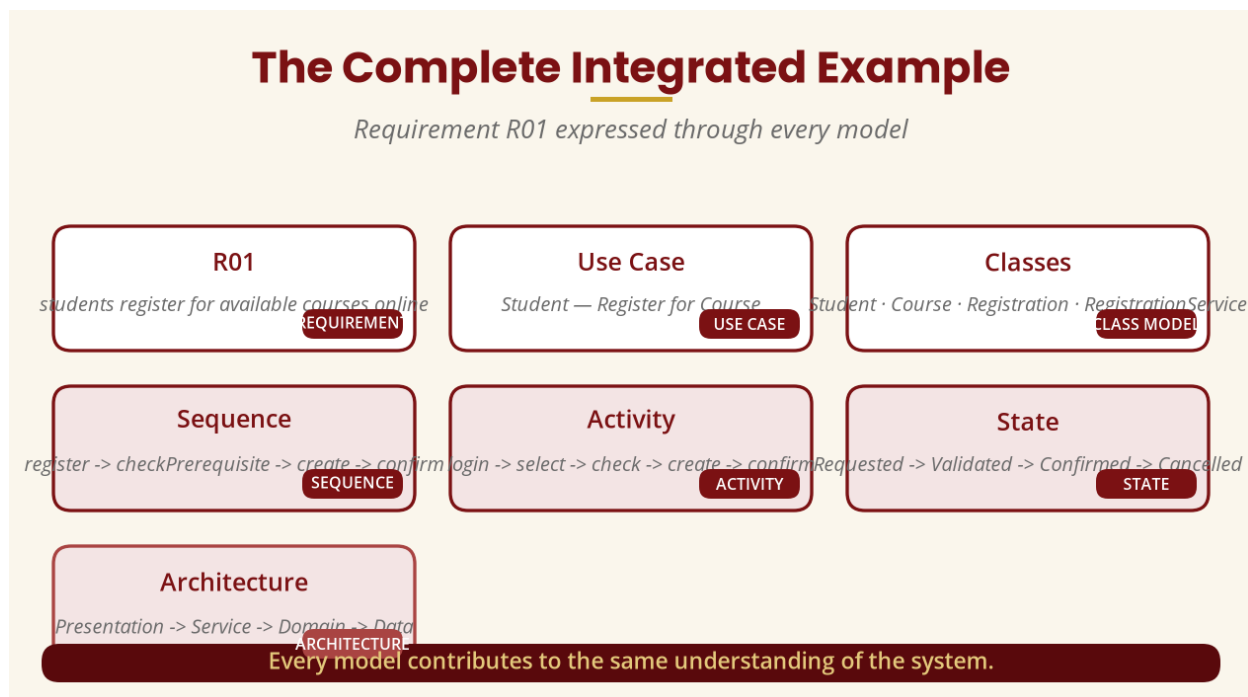


Figure 11 — The complete integrated example

PART J — MODEL CONSISTENCY CHECKING

13. The Consistency Checklist

- **Actors** — does every important actor in the requirements/use cases appear appropriately elsewhere?
- **Use cases** — does every important use case have supporting behaviour?
- **Classes** — do the classes required by interactions exist?
- **Operations** — if the sequence says `course.checkCapacity()`, does Course have that operation?
- **Attributes** — if a requirement needs course capacity, does Course represent it?
- **States** — if the business distinguishes Pending/Approved/Rejected, does the state model represent them?
- **Relationships** — does the class model represent the right multiplicities?

PART K — MULTIPLICITY

14. Why Multiplicity Matters

Student — **Course** is not enough: how many courses can one student take, and how many students can take one course? Modelling the Registration as an entity gives precision: **Student 1 — 0..* Registration** and **Course 1 — 0..* Registration**.

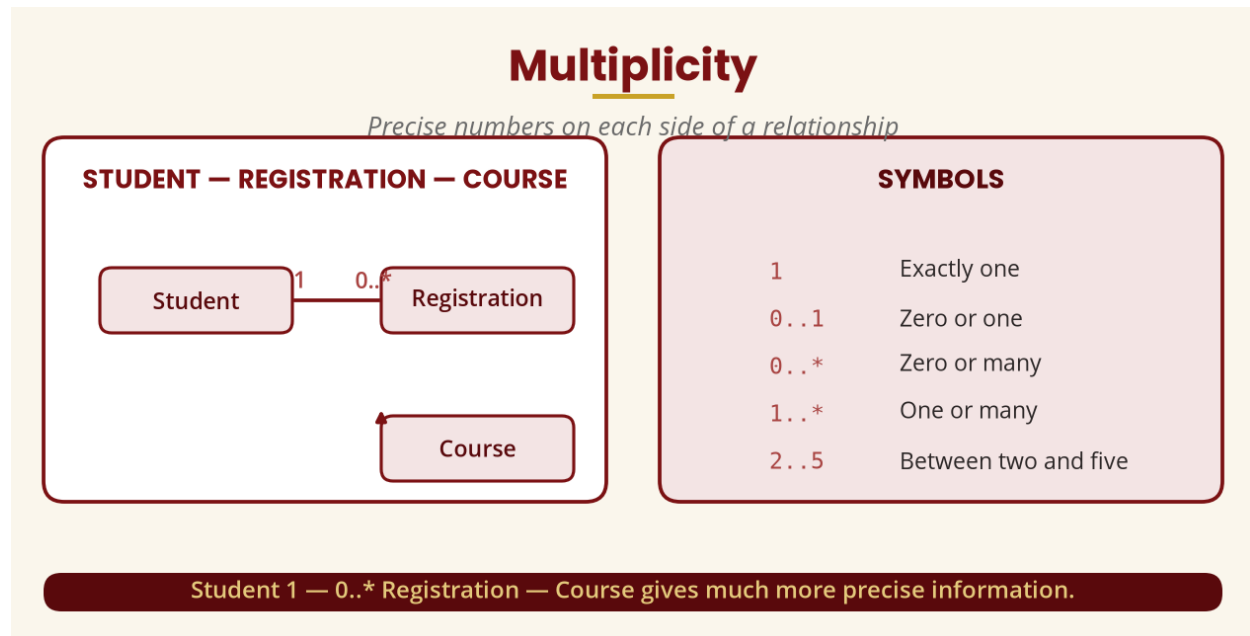


Figure 12 — Multiplicity

Symbol	Meaning
1	Exactly one
0..1	Zero or one
0..*	Zero or many
1..*	One or many
2..5	Between two and five

PART L — PACKAGES: ORGANISING A LARGE MODEL

15. Why Packages?

A large university system may contain Student, Course, Registration, Payment, Invoice, Lecturer, Department, LibraryBook, Borrowing, Notification, User, Role, Permission and Report. Putting everything into one diagram becomes unreadable — so we group related elements into **packages** such as Authentication, Academic, Finance, Library and Notification. UML supports package and other structural modelling constructs as part of its standard modelling language.

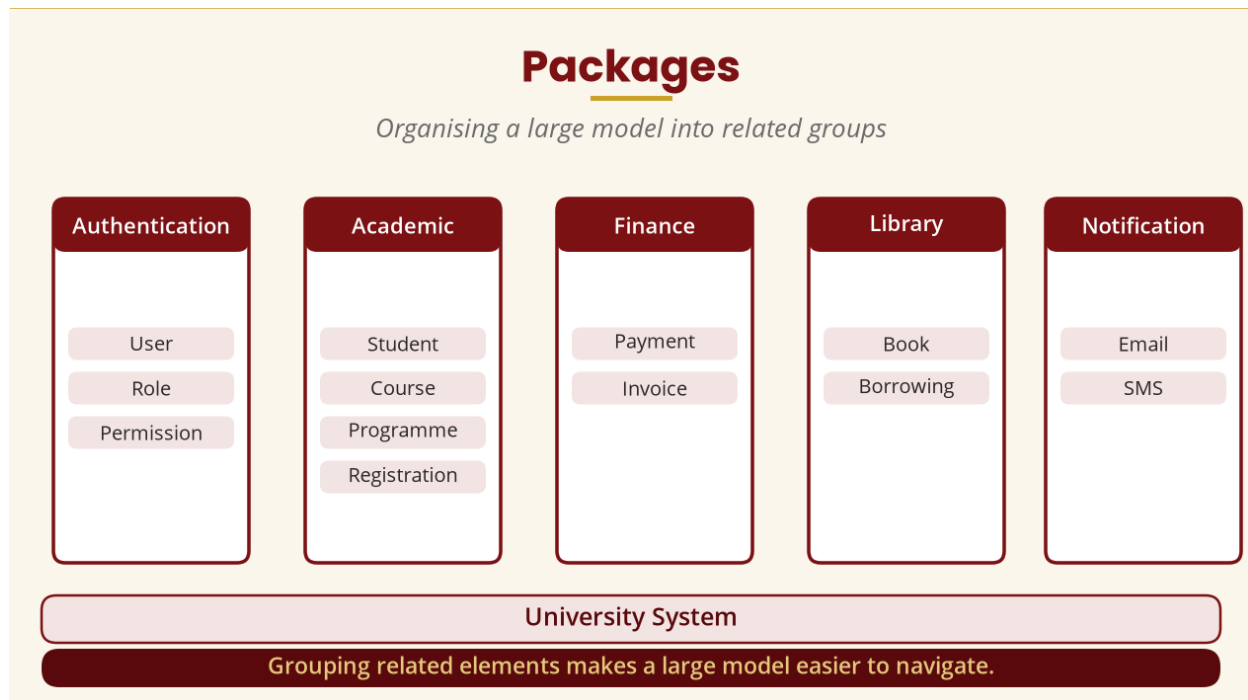


Figure 13 — Package structure

PART M & N — ARCHITECTURAL INTEGRATION AND COMPONENTS

16. Layered Architecture

Moving beyond individual classes, a layered architecture organises the system into Presentation, Application (RegistrationService, AuthenticationService), Domain (Student, Course, Registration), Data Access (repositories) and Database. Everything inside one StudentRegistrationSystem class — HTML, database queries, business rules, authentication, registration, email, reports — becomes hard to maintain; layers give responsibilities clearer boundaries.

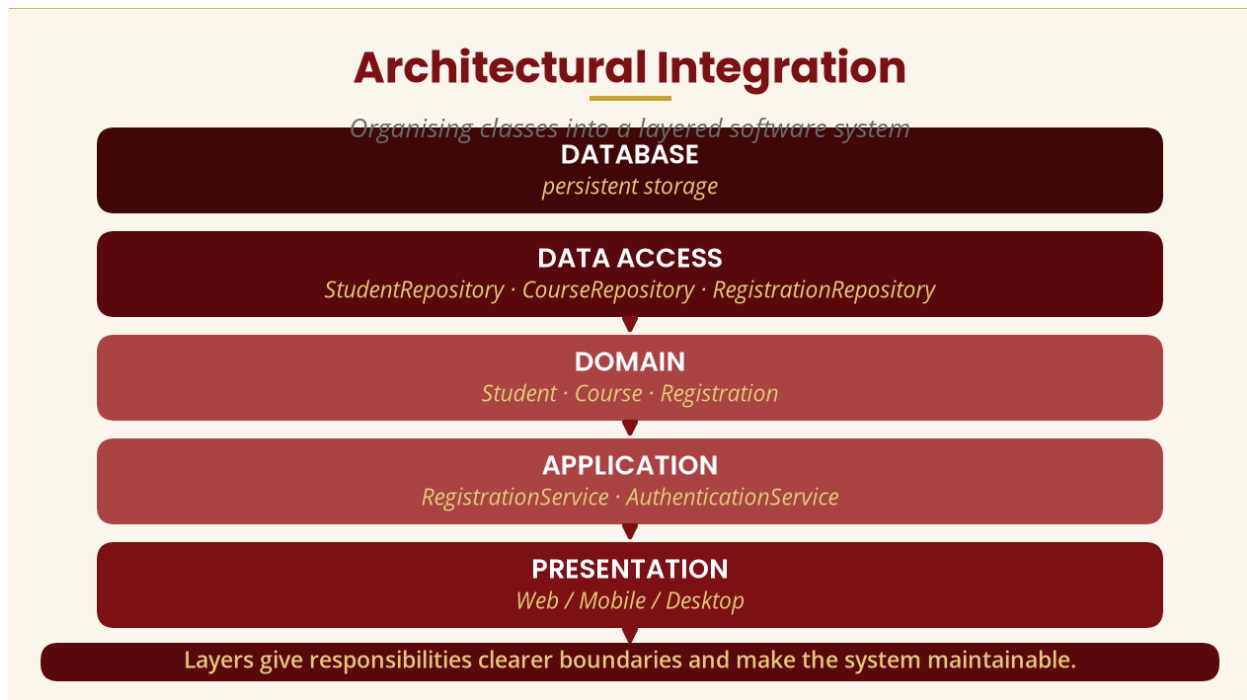


Figure 14 — Layered architecture

17. Component Thinking

A **component** is a modular part of a system that provides or requires functionality — e.g. an Authentication Component, Registration Component, Payment Component, Notification Component and Reporting Component. This asks not simply “what classes exist?” but “how can the software be divided into meaningful, replaceable or independently understandable parts?” A Payment Component with `processPayment()` and `refundPayment()` can communicate with Bank, Mobile Money and Card gateways through an abstraction, keeping the larger system less dependent on a specific implementation.

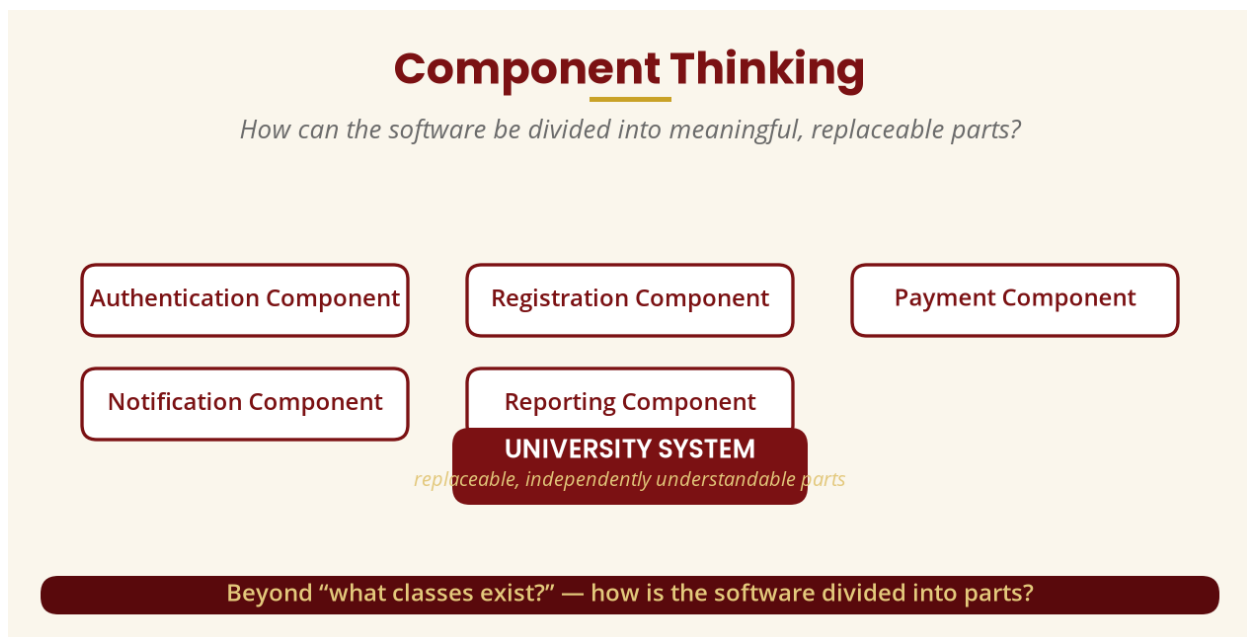


Figure 15 — Component thinking

PART O & P — VALIDATION AND DESIGN REVIEW

18. Design Validation

Validation asks “Does our design actually represent what the system is supposed to do?” If the requirement says “*students cannot register for a full course*” but the sequence shows Student -> RegistrationService -> Create Registration with no `checkCapacity()`, the design is incomplete. The fixed design inserts `Course.checkCapacity()` (and likewise `checkPrerequisite()` for the prerequisites rule).

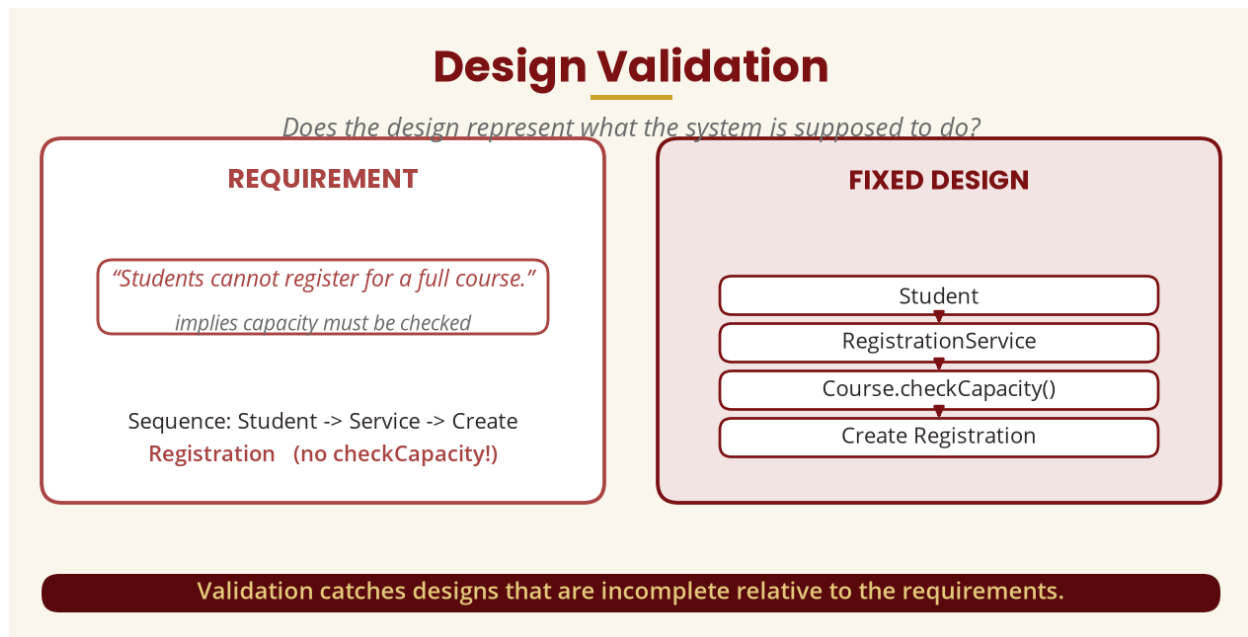


Figure 16 — Design validation

19. Questions a Design Team Should Ask

- **Requirement coverage** — have we represented every important requirement?
- **Use-case coverage** — does every major use case have supporting design elements?
- **Class / interaction / state coverage** — do classes, collaborations and lifecycles cover the requirements?
- **Architecture** — are responsibilities divided appropriately?
- **Coupling / cohesion** — are components unnecessarily dependent? does each have a meaningful focus?
- **Reuse / maintainability** — can components be reused, and what happens when requirements change?

PART Q — MODEL vs DIAGRAM

20. An Important Distinction

“Model” and “diagram” are not synonyms. A **diagram** is a visual representation; a **model** is the underlying structured representation of the system being described — you may have multiple diagrams representing different aspects of the same model. Google Maps offers satellite, roads, terrain, traffic and public-transport views of the same geography; similarly the use-case, class, sequence, state and activity diagrams provide different perspectives on the same software system.

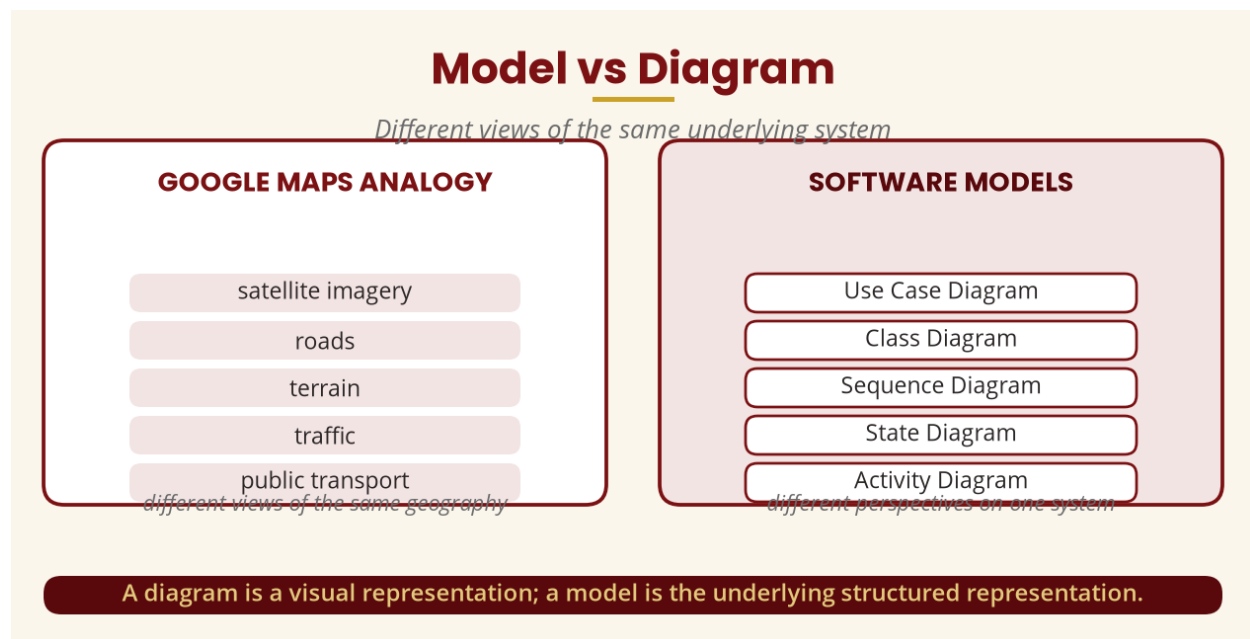


Figure 17 — Model vs diagram

PART R — THE COMPLETE INTEGRATED EXAMPLE

R01: students should be able to register for available courses online. **Use case:** Student — Register for Course. **Classes:** Student, Course, Registration, RegistrationService. **Attributes:** Student (studentId, name, email); Course (courseCode, title, capacity); Registration (registrationId, date, status). **Operations:** Student.registerCourse(); Course.hasAvailableSpace(), checkPrerequisite(); Registration.confirm(), cancel(); RegistrationService.register(). **Sequence:** Student -> register(course) -> RegistrationService -> checkPrerequisite() -> Course -> available? -> create() -> Registration -> confirm() -> Student. **Activity:** login -> select course -> check prerequisite -> check capacity -> create -> confirm. **State:** Requested -> Validated -> Confirmed -> Cancelled. **Architecture:** Presentation -> RegistrationService -> domain -> repositories -> database.



Figure 18 — The Week 12 mental model

21. Tutorial and Practical

Tutorial 1 — For “A patient can select a doctor, choose an available date and time, and book an appointment”, identify actors, use cases, classes, attributes, operations, relationships and possible appointment states. **Tutorial 2** — Explain the question answered by each of the use-case, class, sequence, activity and state diagrams. **Tutorial 3** — A sequence diagram contains PaymentService, Payment and Invoice but the class diagram only has Student, Course and Registration: what inconsistency exists, what should the designer do, and why must the models agree?

Practical (1 hour): create a complete integrated UML model for the Online University Course Registration System — (1) use-case diagram (Student, Administrator), (2) class diagram (Student, Course, Registration, Programme, User, RegistrationService with attributes, operations, visibility, relationships, multiplicities), (3) sequence diagram for registering, (4) activity diagram for the workflow, (5) state diagram for the Registration lifecycle, (6) package diagram (Authentication, Academic, Registration, Administration), and (7) architecture (Presentation, Application, Domain, Data Access, Database).

22. Common Student Confusions

- **Use case vs sequence** — use case = what the user wants to achieve; sequence = how objects communicate to achieve it.
- **Activity vs state** — activity = what activities happen in a process; state = what states a particular object passes through.
- **Class vs object diagram** — class = the blueprint (Student, Course, Registration); object = the actual instance (student:Student, course:Course).

23. Week 12 Summary

Model	Main question
Use case	What does the actor want to accomplish?
Class	What objects/classes make up the system?
Sequence	How do objects interact over time?
Activity	What workflow/activities take place?
State machine	How does an object change state?
Component / deployment	How is the software organised and deployed?

THE CENTRAL PRINCIPLE OF WEEK 12

Every model should contribute to the **same understanding of the system**. Requirements -> use cases -> what the system must do -> behavioural models -> how the behaviour occurs -> object/class model -> what structure supports the behaviour -> architecture -> how the software is organised -> implementation. **Don't memorise UML diagrams as independent pictures — learn to tell the same system's story through different models, and make sure those models agree.**

24. Examination–Style Questions

Essay (25 marks). Explain integrated object-oriented modelling. Using an Online University Registration System, demonstrate how the use-case, class, sequence, activity and state models can be integrated into a coherent design.

Essay (20 marks). “Different UML diagrams represent different views of the same software system.” Explain this statement with examples.

Essay (20 marks). A sequence diagram contains checkAvailability() but the class diagram has no such operation. (a) Identify the modelling problem. (b) Explain why consistency matters. (c) Describe how the designer should resolve it.

Scenario (25 marks). Design an integrated UML model for an online hospital appointment system: actors, use cases, classes, relationships, sequence of interactions, activities, appointment states, and an explanation of how the models are consistent.

25. References and Further Reading

Authoritative / current online sources

- Object Management Group (OMG). *Unified Modeling Language (UML), Version 2.5.1*. OMG — the formal UML specification and machine-readable UML artefacts.
- Object Management Group. *UML — Unified Modeling Language*. Overview of UML's purpose in visualising, specifying and documenting software and systems.

-
- Object Management Group. *UML 2.5.1 Specification — Technical Specification*. Structural and behavioural modelling clauses and UML diagram concepts.

Prescribed and recommended texts

- Mach, E. (2019). *Object Oriented Analysis & Design Cookbook: Introduction to Practical System Modeling*.
- Reddy, V., & Korra, S. (2018). *Object-Oriented Analysis and Design Using UML*.
- The Art of Service. (2021). *Object Oriented Analysis and Design: A Complete Guide*.
- Wixom, B., & Dennis, A. (2018). *Systems Analysis and Design*.
- Blokdyk, G. (2021). *Object-Oriented Analysis and Design Standard Requirements*.